Ride-Hailing Order and Trip Dispatch Database

(Topic 5)

CDS 534 Database Management — Group Project

Group: **Royal Family**

A · Business & Requirements	CHEN Yinan (3162550)
B · ER Modelling	ZHANG Yijie (3162641)
C · Implementation	CHEN Yanwu (3162744)
D · Queries & Experiments	LING Tong (3162677)
E · PM & Integration	HU Zhongyuan (3162304)

November 10, 2026

Contents

1 Business Context and Goals **2**
- 1.1 Background and Business Process . . . 2
- 1.2 Stakeholders and Their Data Needs . . . 2
- 1.3 Quantified Goals, Constraints and Trade-offs . . . 2
- 1.4 Dataset Selection Rationale . . . 3

2 Entity-Relationship Model **4**
- 2.1 Notation and Conventions . . . 4
- 2.2 Official Five-Entity Base . . . 4
- 2.3 Approved Extensions . . . 5
- 2.4 Normalization . . . 5

3 Implementation **6**
- 3.1 Environment and Architecture . . . 6
- 3.2 DDL Organisation and Key Constraints . . . 6
- 3.3 Data Generation and Import . . . 7
- 3.4 Verification . . . 7

4 SQL Queries and Conceptual Q&A **7**
- 4.1 Business Queries . . . 7
- 4.2 Conceptual Q&A . . . 11

5 Evaluation **13**
- 5.1 Workload Profile . . . 13
- 5.2 Experiment 1: Index Before/After . . . 13
- 5.3 Experiment 2: JOIN vs Subquery . . . 14
- 5.4 Experiment 3: Concurrent Transactions . . . 14
- 5.5 Distribution Check against the RideAustin Holdout . . . 15
- 5.6 Results Summary and Goal Recheck . . . 15

6 Conclusion and Limitations **16**
- 6.1 Goal Recheck . . . 16
- 6.2 Limitations . . . 16
- 6.3 Future Work . . . 17

A Contribution Statement **18**

B Full Build Script **18**

1 Business Context and Goals

1.1 Background and Business Process

This project models a ride-hailing platform from dispatch to trip completion, payment and optional rating. The official schema contains five base entities, `Passenger`, `Driver`, `Vehicle`, `Trip` and `Payment`. The project retains these five tables and adds `trip_detail`, `service_tier` and `pickup_zone`, giving eight entities in total. The foreign key, `NOT NULL` and `UNIQUE` constraints on `Driver.vehicle_id` assign each driver one vehicle and each vehicle at most one driver. Similarly, `Payment.trip_id` is a unique foreign key, so each trip can have at most one payment.

`trip_detail` links each trip to its service tier and pickup zone and stores an optional rating. Its primary key, `trip_id`, also serves as a foreign key to `Trip`. Two further foreign keys reference `service_tier` and `pickup_zone`. The generator creates one detail record per trip, including cancelled trips. The primary key and foreign key constrain each trip to at most one detail record.

Under decision D-05, request matching takes place in the application layer. A `Trip` row is inserted only after successful dispatch, with `start_time` recording that time. Unmatched requests are therefore unavailable for analysing total ride demand or matching latency (Section 6). Trip status progresses through `accepted`, `en_route` and `completed`. Early termination after boarding is classified as `completed`. Under D-06, cancellation before boarding is free within the grace period. Late cancellation or a passenger no-show incurs a $1.50 fee, subject to a waiver for excessive pickup delays. Each charge, including a cancellation fee, is recorded in `Payment` [4, 3].

1.2 Stakeholders and Their Data Needs

Table 1 lists two requirements for each stakeholder group. Trip reconciliation and vehicle assignment motivate the foreign key and uniqueness constraints discussed in Section 2. The customer-service requirement to retrieve a complete trip record within 30 seconds provides the business rationale for query-latency goal G2.

1.3 Quantified Goals, Constraints and Trade-offs

Table 2 distinguishes project design targets from the available verification evidence. The 200 ms latency target, provisional 50 TPS throughput target and 180-day retention period are project requirements. Constraints include 180-day online retention with archiving (C1), relationship cardinalities enforced through UNIQUE constraints (C2), non-negative payment amounts enforced through CHECK (C3), compatibility with the group's lowest MySQL version, 8.0.45 (C5), and rebuilding within 30 minutes using repository scripts and a fixed random seed (C6).

Workload assumption for G3. Assuming 50,000 trips per day, with 20% occurring within two to four hours, the estimated arrival rate is 0.69–1.39 trips per second. Converting this rate to database TPS requires defining the transactions and their boundaries for each trip. The 50 TPS target remains unvalidated.

Consistency trade-off. Dispatch and payment writes require strong consistency, while customer-service lookups require the latest trip status. Transactions and FK, UNIQUE and CHECK constraints protect data integrity. Aggregate reads, including

Table 1: Stakeholders, data requirements and supporting tables

Stakeholder	Data requirement	Supporting tables and keys
Passengers	Trip and fare history	`Trip.psg_id`; `Payment.trip_id`
	Fare and payment reconciliation	`Trip.fare`, `Payment.amount`
Drivers	Cumulative trips and earnings	`Trip.driver_id`, `SUM(fare)`
	Unique, verifiable vehicle assignment	`Driver.vehicle_id` (FK, UNIQUE)
Operations	Peak trip volume and cancellation rate	`Trip.start_time`, `Trip.status`
	Driver and vehicle assignment audit	`Driver` joined to `Vehicle`
Finance	Trip reconciliation and zero orphan payments	`Payment.trip_id` (FK, UNIQUE); anti-join for unpaid completed trips
	Daily/monthly payment totals by method	`SUM(Payment.amount)`, `Trip.start_time`, `Payment.method`
Customer service	Retrieve complete trip records within 30 s	Join `Passenger`, `Trip`, `Driver`, `Vehicle` and `Payment`
	Verify disputed fares	`Trip.fare`, `Payment.amount`, `Payment.method`

reconciliation reports, monthly revenue summaries and the five-minute dispatch dashboard, can tolerate minute-level staleness. The A5 workload model estimates that writes account for approximately 99% of statements, while aggregate reads account for over 95% of scanned rows. These estimates motivate routing aggregate queries to a read replica or cache. The project runs on a single MySQL instance, so replication remains a design proposal (Section 6).

1.4 Dataset Selection Rationale

`import_data.py` generates 60,000 trips using a fixed random seed, allowing all members to rebuild the same dataset (C6, D-08). RideAustin [5, 6], a CC0-licensed dataset of approximately 1.49 million trips in Austin during 2016–2017, provides calibration data for service tiers, fares, rating coverage and the evening peak (17:00–20:00). An 80/20 calibration/holdout split supports the comparison in Section 5 (D-12). The generated REGULAR/SUV/PREMIUM/LUXURY shares are 95.1/3.7/0.9/0.3%, with approximately 90% rating coverage on completed trips. Section 5 reports an evening-peak share of 22.4% in the generated data and approximately 20.5% in the holdout.

Field compatibility motivates using synthetic data. NYC TLC high-volume FHV records [2] lack the passenger identifiers, payment methods and driver–vehicle assignments required by the schema. RideAustin provides ratings, service tiers and passenger/-driver identifiers, making it suitable for calibration. The sample spans 365 days for query demonstrations, while 180-day online retention remains a design constraint. It intentionally includes 1,442 completed trips without payment for reconciliation tests. Cancellation rules, payment methods, the unpaid proportion and 15 pickup zones (top three approxi-

Table 2: Quantified design goals and verification methods at sample scale

#	Design goal	Verification method
G1	At least 50,000 trips	Row-count checks after import (delivered: 60,000)
G2	Top-5 query latency $P95 <$ 200 ms (Q6, Q1, Q5, Q2, Q7)	Experiment 1 compares single-run EXPLAIN ANALYZE results before/after indexing on representative `status` and `start_time` predicates. Top-5 P95 remains unmeasured.
G3	Provisional write-throughput target of approximately 50 TPS	Requires a defined transaction workload and load testing. Experiment 3 demonstrates concurrency behaviour, without measuring throughput.
G4	Zero missed `completed` trips without payment, full-table execution < 500 ms	Experiment 2 compares full-table anti-join results and execution times. Zero missed trips requires independent verification of expected trip IDs.
G5	Referential integrity with zero orphan rows, violations rejected by the database	Experiment 3 demonstrates FK error 1452, UNIQUE error 1062 and CHECK error 3819.

mately 35%) are course assumptions. These dimensions are excluded from the RideAustin validation.

2 Entity-Relationship Model

2.1 Notation and Conventions

Figure 1 shows our eight-entity model. As required by the course handout [1], rectangles contain attributes, primary keys are italic, foreign keys carry an asterisk, and every link has a cardinality label. There is no direct many-to-many link. The five official entities remain unchanged; three separate extension tables implement the group's approved pickup-zone and service-tier questions (D-11).

2.2 Official Five-Entity Base

`Passenger` and `Driver` each link to many `Trip` rows through the required `psg_id` and `driver_id` foreign keys. Under D-05, a Trip row starts when a driver accepts the request; `start_time` therefore represents acceptance, not passenger boarding. Each `Driver` references one `Vehicle`. The official `Driver.vehicle_id UNIQUE` constraint also limits a vehicle to at most one recorded driver, despite the broader N:1 label in the teacher's sample figure. Each `Payment` references one Trip, while `Payment.trip_id UNIQUE` permits zero or one payment per Trip. The zero case supports the official query for completed but unpaid trips. These are the actual cardinalities enforced by Listing 17, not additional assumptions.

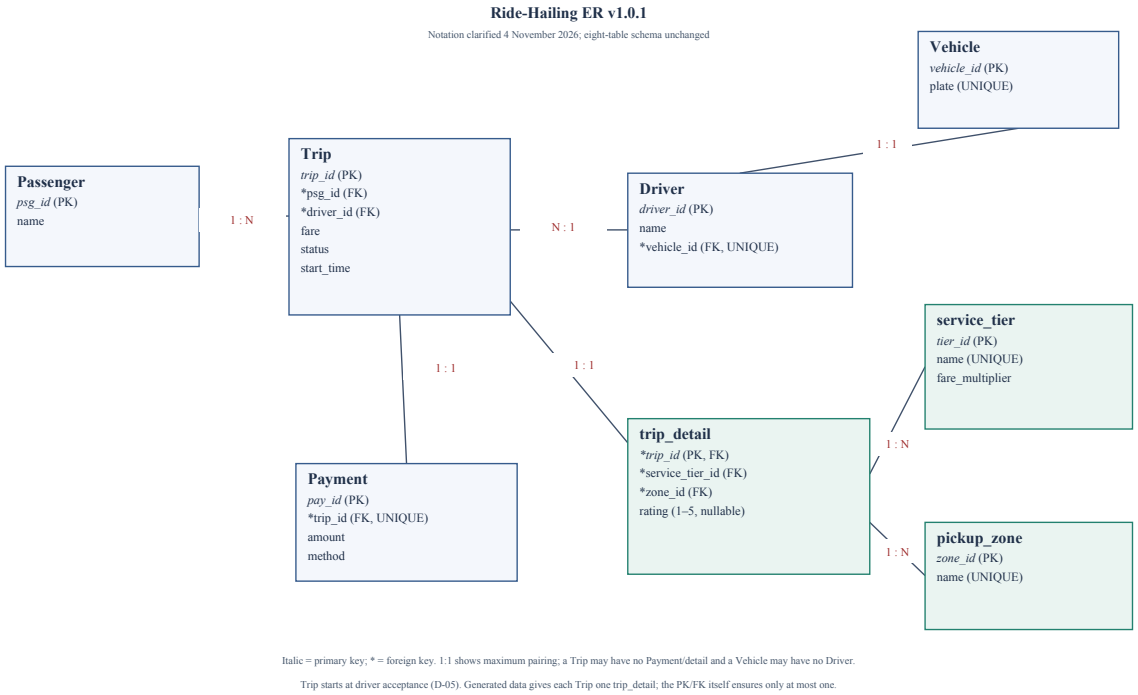


Figure 1: Final ER v1.0.1 (notation clarified; the eight-table schema is unchanged). Blue boxes are the five official Topic 5 tables; green boxes are the D-11 extensions. The course-style 1:1 labels show the maximum pairing; optional participation is stated in the text and diagram note.

2.3 Approved Extensions

The `pickup_zone` table gives each accepted trip one named pickup zone through `trip_detail.zone_id`. It supports q09, which counts all accepted Trip rows, including later cancellations; it cannot measure unmatched requests or unmet demand. The `service_tier` table stores the four original RideAustin tier names and a descriptive fare multiplier. Its link through `trip_detail.service_tier_id` supports q10: completed-trip count, average fare, valid per-trip rating count and average rating by tier. The rating is the passenger's rating of this trip, not a driver's historical average. These two dimensions and the nullable rating live in `trip_detail`, whose `trip_id` is both primary and foreign key. Thus the SQL allows at most one detail row per Trip; the fixed-seed generator supplies exactly one. The generator also keeps ratings on completed trips only. Those cross-table rules are checked in data validation, not enforced by the rating-range `CHECK` alone.

2.4 Normalization

We checked all eight tables using the 1NF–3NF tests taught in Lecture 08. Each column holds one value, so the tables satisfy 1NF. Every declared candidate key is a single column; no non-key attribute can depend on only part of a composite key, so they satisfy 2NF. Names of passengers, drivers, tiers and zones are stored in their own entity tables rather than repeated with every Trip; under the stated business dependencies, no non-key attribute depends transitively on a key within one table, so the present schema satisfies 3NF. The detailed table-by-table check is recorded in `01_docs/b5_normalization_review.md`.

For a concrete 3NF example, suppose an unnormalised `TripRaw` also stored `psg_name`. The dependencies would be `trip_id` → `psg_id` → `psg_name`. Since one passenger may take many trips, `psg_id` is not a key of `TripRaw`; the name would be repeated and could be updated inconsistently. Our existing `Passenger(psg_id, name)` and `Trip(trip_id, psg_id, ...)` tables remove that transitive dependency while keeping the link through a foreign key. No change to the teacher's five-table DDL is needed.

3 Implementation

The design of Section 2 is realised by three repository artefacts: the idempotent build script `02_sql/build.sql` (§3.2), the deterministic data generator `02_sql/import_data.py` (§3.3), and a clean-room acceptance procedure with a standardised screenshot package (§3.4).

3.1 Environment and Architecture

All five members work on a local MySQL 8 instance under one shared schema, `ride_hailing` (`utf8mb4_general_ci`); the group compatibility baseline is the lowest registered server version (8.0.45), so the scripts avoid CTEs and window functions and treat pre-8.0.16 servers as silently ignoring CHECK constraints. GitHub is the single source of truth: a schema change counts only when it lands in `build.sql` ("change the table = change the script = commit"), and after every pull members rebuild the database from scratch, keeping the five environments identical by construction. Screenshot evidence follows a written specification (naming `prefix_object_MMDD.png`; whole-window captures keeping the statement, the result grid and the action-output panel in one frame), so every figure in this report traces back to the exact machine, script and statement that produced it.

3.2 DDL Organisation and Key Constraints

`build.sql` is organised in four blocks: (0) drop/recreate the schema; (1) the official five-table DDL of Topic 5, kept unmodified in the repository ("base immutability rule"); (2) the official seed queries; (3) the extension zone; (4) a self-check block printing the server version, the table list and per-table counts, turning every rebuild into a verifiable event. Extensions never touch the five base tables: two seeded dimension tables (`service_tier`, four tiers calibrated to RideAustin shares; `pickup_zone`, 15 zones) and one 1:1 extension table `trip_detail` whose primary key is also a foreign key to `Trip`. Table 3 maps each ER cardinality to the constraint that enforces it.

Table 3: ER cardinalities and their SQL enforcement (`build.sql`).

Relationship	Constraint	Cardinality
Trip–Payment	`Payment.trip_id NOT NULL UNIQUE` + FK	1:(0..1)
Vehicle–Driver	`Driver.vehicle_id NOT NULL UNIQUE` + FK	1:(0..1)
Passenger/Driver–Trip	NOT NULL FKs	1:N mandatory
ServiceTier/PickupZone–TripDetail	NOT NULL FKs	1:N mandatory
Trip–TripDetail	`trip_id` PK-as-FK	1:(0..1)

One design rule cannot be expressed in DDL: `rating` must be non-NULL only for completed trips — a cross-table condition that CHECK cannot reference. It is enforced by the generator and documented as a deliberate constraint-expressiveness boundary; trigger-based enforcement was considered out of course scope.

3.3 Data Generation and Import

The generator uses the Python standard library only — no third-party dependencies — so all five machines produce identical output regardless of installed packages. A single seeded RNG (`seed=534`) drives exact-composition pools: each categorical attribute (status, payment method, tier mix, rating) is drawn from a pool with exact counts and then shuffled, so proportions are exact and only the order is random. Fares follow the group's RideAustin-calibrated formula — base + per-minute + per-mile + a fixed $1.65 surcharge; the REGULAR tier uses a two-period mile tariff (0.99/1.50 per mile, 80/20 split) — and free cancellation (no payment row), charged cancellation ($1.50) and completed-but-unpaid trips are all represented per the fare-calibre decision. Determinism was verified by generating the import file twice and comparing checksums: the 174,698-statement `data_import.sql` is byte-identical across runs and machines. Table 4 lists the resulting row counts.

Table 4: Row counts after a clean rebuild and import (`seed=534`).

	Passenger	Vehicle	Driver	Trip	Payment	service_tier	pickup_zone	trip_detail
Rows	3,000	150	150	60,000	51,398	4	15	60,000

3.4 Verification

Acceptance was defined as a clean-room procedure: drop the schema, run `build.sql`, import `data_import.sql`, then execute the twelve-statement acceptance script `02_sql/c5_check.sql`. It re-derives the eight table counts of Table 4, runs zero-anomaly checks (no orphaned foreign keys, no trip without an extension row, no payment for a non-existent trip), and spot-checks business queries against pre-computed expected values: the first Q1 row (trip 1, Justin Lewis, fare 8.33, cash), exactly 150 drivers with completed-unpaid trips (Q2), and the four-tier aggregate of q10. Every check passed with observed values identical to expected. Evidence is captured per the screenshot specification: 21 verification screenshots (2026-11-01) — structure and row count for each of the eight tables, the row-count summary (Fig. 2), and four query spot-checks — plus the ten query-result screenshots referenced in Section 4. The procedure simultaneously closes goal G1 (60,000 trips $\geq$ 50,000) and demonstrates goal G5 (constraint enforcement; error-code demonstrations appear with the experiments in Section 5).

4 SQL Queries and Conceptual Q&A

4.1 Business Queries

Ten queries answer the workload, all executed on the 60,000-trip sample. Each listing below is the exact SQL from `02_sql/queries/`; screenshots are archived in the repository.

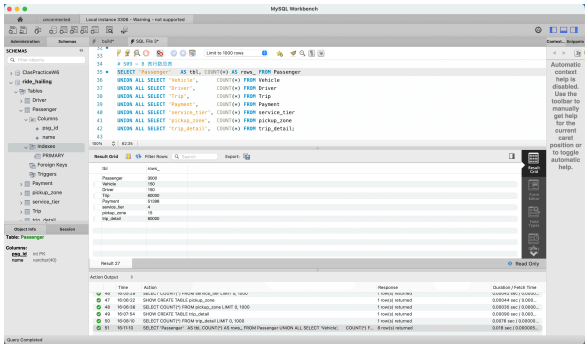


Figure 2: Eight-table row-count self-check.

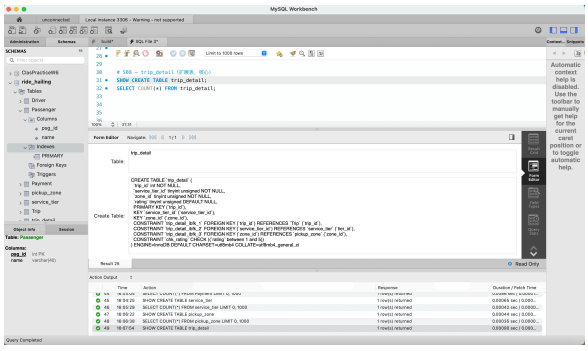


Figure 3: `trip_detail` DDL: PK-as-FK, three FKs, rating CHECK.

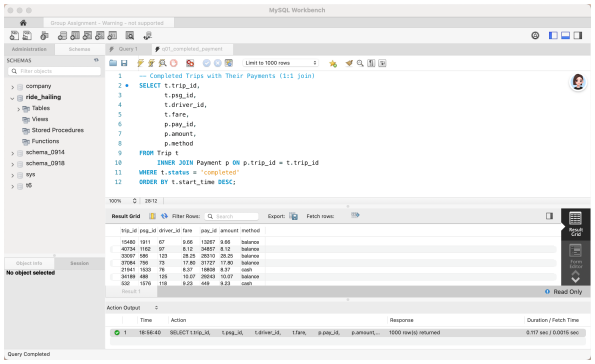
Q1–Q2: Payment reconciliation and collections. Q1 pairs every completed trip with its payment via the UNIQUE `trip_id` join — `fare` equals `amount` row by row. Card is the dominant payment method. Q2 flips the join into a `NOT EXISTS` anti-join to find drivers with uncollected fares: 150 drivers, 1,442 trips. Cancelled trips carrying the $1.50 fee already have a Payment row, so they stay off the list.

Listing 1: Q1: completed trips with payment.

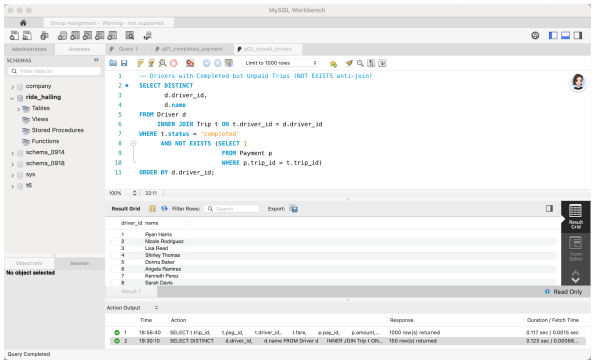
```
SELECT t.trip_id, t.psg_id, t.driver_id, t.fare,
       p.pay_id, p.amount, p.method
FROM Trip t
     INNER JOIN Payment p ON p.trip_id = t.trip_id
WHERE t.status = 'completed'
ORDER BY t.start_time DESC;
```

Listing 2: Q2: completed but unpaid drivers.

```
SELECT DISTINCT d.driver_id, d.name
FROM Driver d
     INNER JOIN Trip t ON t.driver_id = d.driver_id
WHERE t.status = 'completed'
      AND NOT EXISTS (SELECT 1
                      FROM Payment p
                      WHERE p.trip_id = t.trip_id)
ORDER BY d.driver_id;
```



(a) Q1 (first 1,000 of ≈49.2k rows).



(b) Q2: 150 drivers, 1,442 unpaid trips.

Q3–Q4: Revenue by vehicle and by driver-day. Q3 aggregates across `Vehicle`→`Driver`→`Trip`; revenue is balanced (head vehicles 440–460 trips vs. fleet average 400). Q4 ranks driver-

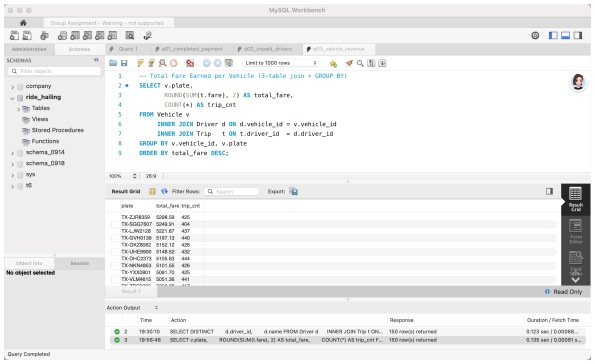
days with `RANK() OVER` on the aggregated daily income — one pass, no self-join. The top earner makes $292 in a single day.

Listing 3: Q3: total fare per vehicle.

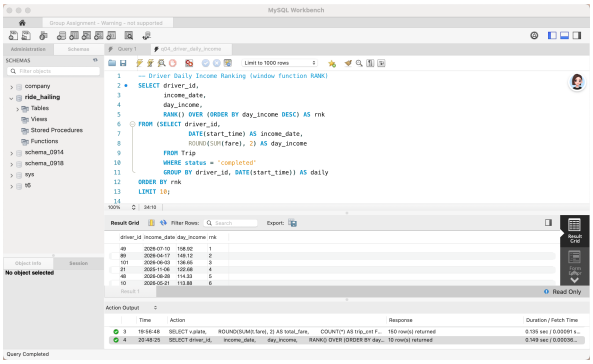
```sql
SELECT v.plate,
       ROUND(SUM(t.fare), 2) AS total_fare,
       COUNT(*) AS trip_cnt
FROM Vehicle v
     INNER JOIN Driver d ON d.vehicle_id = v.vehicle_id
     INNER JOIN Trip   t ON t.driver_id  = d.driver_id
GROUP BY v.vehicle_id, v.plate
ORDER BY total_fare DESC;
```

Listing 4: Q4: driver daily income ranking.

```sql
SELECT driver_id, income_date, day_income,
       RANK() OVER (ORDER BY day_income DESC) AS rnk
FROM (SELECT driver_id,
             DATE(start_time) AS income_date,
             ROUND(SUM(fare), 2) AS day_income
       FROM Trip
       WHERE status = 'completed'
       GROUP BY driver_id, DATE(start_time)) AS daily
ORDER BY rnk
LIMIT 10;
```



(a) Q3: 150-vehicle fleet.



(b) Q4: top-10 driver-days.

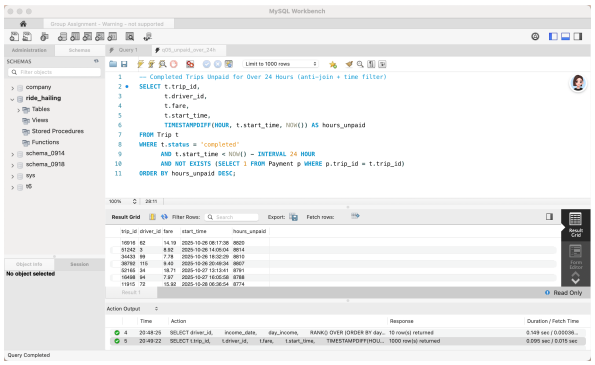
Q5–Q6: Collections exposure and dispatch planning. Q5 filters unpaid completed trips beyond 24 hours — all 1,442 qualify because the simulated history ends before load time; the threshold would isolate overdue receivables in production. Q6 groups trips by hour and keeps hours above 2,000 trips with `HAVING`, something `WHERE` cannot do. The evening block 17:00–20:00 carries 22.4% of volume, peaking at hour 18.

Listing 5: Q5: unpaid >24h.

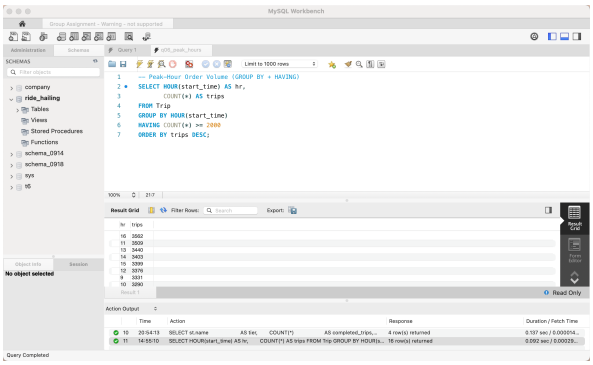
```sql
SELECT t.trip_id, t.driver_id, t.fare, t.start_time,
       TIMESTAMPDIFF(HOUR, t.start_time, NOW()) AS hours_unpaid
FROM Trip t
WHERE t.status = 'completed'
      AND t.start_time < NOW() - INTERVAL 24 HOUR
      AND NOT EXISTS (SELECT 1 FROM Payment p WHERE p.trip_id = t.trip_id)
ORDER BY hours_unpaid DESC;
```

Listing 6: Q6: peak-hour volume.

```sql
SELECT HOUR(start_time) AS hr,
       COUNT(*) AS trips
FROM Trip
GROUP BY HOUR(start_time)
HAVING COUNT(*) >= 2000
ORDER BY trips DESC;
```



(a) Q5 (first 1,000 of 1,442).



(b) Q6: 14 hours above threshold.

Q7–Q8: Trends and retention. Q7 joins payments to trip months: revenue climbs from \$49.3k to \$54.8k over twelve months, about +11% with no dip. Q8 keeps passengers active on two or more distinct days — the heaviest rides 79 trips across 74 days.

Listing 7: Q7: monthly revenue.

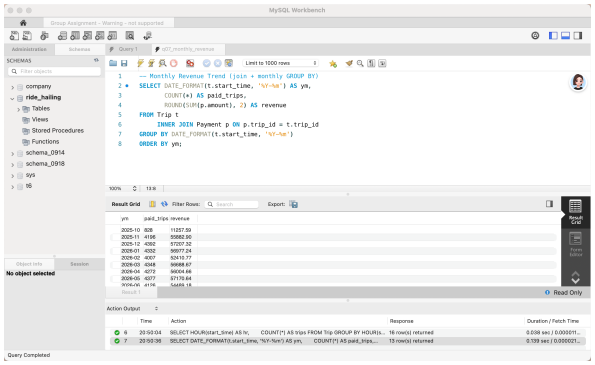
```sql
SELECT DATE_FORMAT(t.start_time, '%Y-%m') AS ym,
       COUNT(*) AS paid_trips,
       ROUND(SUM(p.amount), 2) AS revenue
FROM Trip t
     INNER JOIN Payment p ON p.trip_id = t.trip_id
GROUP BY DATE_FORMAT(t.start_time, '%Y-%m')
ORDER BY ym;
```

Listing 8: Q8: repeat passengers.

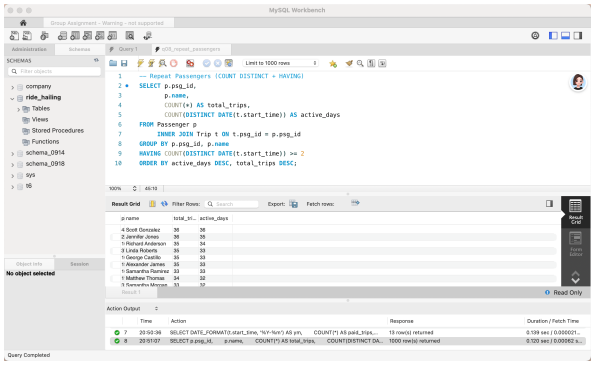
```sql
SELECT p.psg_id, p.name,
       COUNT(*) AS total_trips,
       COUNT(DISTINCT DATE(t.start_time)) AS active_days
FROM Passenger p
     INNER JOIN Trip t ON t.psg_id = p.psg_id
GROUP BY p.psg_id, p.name
HAVING COUNT(DISTINCT DATE(t.start_time)) >= 2
ORDER BY active_days DESC, total_trips DESC;
```

Q9–Q10: Spatial and product-line views from the extension tables. Q9 joins through `trip_detail` to `pickup_zone`: the top three zones absorb 34.6% of accepted trips (Downtown 16.9%, West Campus 9.4%, South Congress 8.2%). This counts accepted trips only — requests without a driver are outside the data. Q10 joins to `service_tier` and uses `COUNT(rating)` for coverage (NULL-skipping) and `AVG(rating)` for the mean: fare climbs 3× across tiers while ratings stay flat at 4.85.

Listing 9: Q9: accepted trips per zone.



(a) Q7: twelve months.

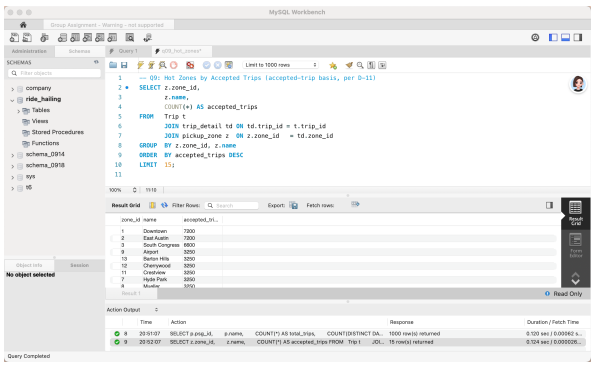


(b) Q8 (first 1,000 rows).

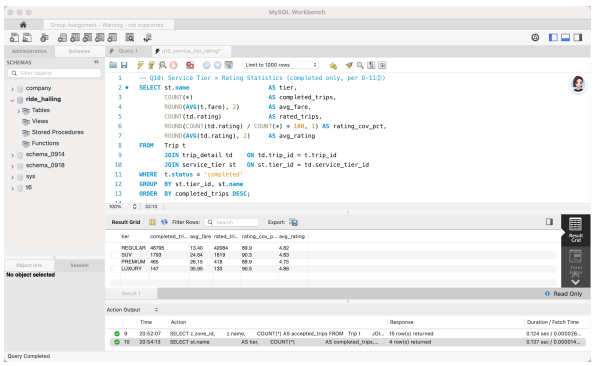
```sql
SELECT z.zone_id, z.name,
       COUNT(*) AS accepted_trips
FROM Trip t
     JOIN trip_detail td ON td.trip_id = t.trip_id
     JOIN pickup_zone z  ON z.zone_id   = td.zone_id
GROUP BY z.zone_id, z.name
ORDER BY accepted_trips DESC
LIMIT 15;
```

Listing 10: Q10: service tier vs. rating.

```sql
SELECT st.name AS tier,
       COUNT(*) AS completed_trips,
       ROUND(AVG(t.fare), 2) AS avg_fare,
       COUNT(td.rating) AS rated_trips,
       ROUND(COUNT(td.rating) / COUNT(*) * 100, 1) AS rating_cov_pct,
       ROUND(AVG(td.rating), 2) AS avg_rating
FROM Trip t
     JOIN trip_detail td  ON td.trip_id = t.trip_id
     JOIN service_tier st ON st.tier_id = td.service_tier_id
WHERE t.status = 'completed'
GROUP BY st.tier_id, st.name
ORDER BY completed_trips DESC;
```



(a) Q9: 15 zones.



(b) Q10: four tiers.

4.2 Conceptual Q&A

Q1: Primary key vs. foreign key. A primary key is NOT NULL and UNIQUE and acts as the address of a row (`Passenger.psg_id`). A foreign key is a child-table column whose values must exist in the parent key (`Trip.psg_id`). MySQL rejects `Trip(psg_id=99999)`

with error 1452 if the passenger does not exist, and blocks deleting a still-referenced parent under RESTRICT (or CASCADE / SET NULL if declared). In our schema `Payment.trip_id` is NOT NULL, UNIQUE and a foreign key at once, so a payment always attaches to exactly one real trip — the 1:1 implementation itself.

Q2: 1NF, 2NF and 3NF. 1NF: each cell holds one atomic value. 2NF: no non-key attribute depends on only part of a composite key — ours use single-column surrogate keys, so 2NF holds by construction. 3NF: no transitive dependencies. If we stored the plate in `Driver(driver_id, name, vehicle_id, plate)`, plate would depend on vehicle_id, which depends on the key. That chain causes update, deletion and insertion anomalies. Splitting into `Driver` and `Vehicle` breaks it, which is the structure we adopted.

Q3: INNER, LEFT and RIGHT JOIN. INNER JOIN keeps only matching rows; LEFT JOIN keeps every left-table row, filling unmatched right columns with NULL. LEFT JOIN is mandatory when zero-trip drivers must appear in a per-driver report — INNER JOIN would drop them silently. The status filter goes in the ON clause (in WHERE it degenerates the join back to INNER), and the count uses `COUNT(t.trip_id)` instead of `COUNT(*)` so unmatched drivers read 0, not 1.

Q4: Subquery vs. JOIN. The same question "who rode vehicle 3" can be written with a JOIN plus DISTINCT, with `psg_id IN (SELECT ...)`, or with EXISTS. In principle the JOIN should win because the optimizer uses join-column indexes in one plan. In practice MySQL 8 rewrites most IN and EXISTS forms into semi-joins, so the plans come out identical. Our Experiment 2 confirms this: all three anti-join variants return 1,442 rows with the same plan, within 8 ms. NOT IN is unsafe if the subquery column can be NULL — `Payment.trip_id` is NOT NULL, so it works here.

Q5: ACID and concurrent payments. Atomicity, consistency, isolation and durability guarantee that transactions either fully succeed or leave no trace, and that committed state satisfies all constraints. With a hypothetical wallet at 100 and two concurrent $50 deductions, both transactions read 100 and write 50 — the classic lost update. This happens because the application reads before updating across two sessions; a single atomic `UPDATE balance = balance - 50` would not lose anything. The default REPEATABLE READ does not prevent it either, since plain SELECTs are non-locking. The fix is two-layered: `FOR UPDATE` serializes the read-modify-write, while the UNIQUE on `Payment.trip_id` blocks double payments at the schema level.

Q6: Indexes and B+ trees. An InnoDB index is a B+ tree whose leaves hold keys with row pointers linked in order, so a lookup descends in O(log n) and range scans read sequentially. The cost: each index takes storage, every write maintains it, and the optimizer rarely uses an index on a low-selectivity column alone. The rule is to index high-selectivity columns in WHERE, JOIN and ORDER BY, respecting the leftmost prefix (status equality before start_time range in our composite index). Foreign-key columns get indexes automatically, which is why our baseline uses (status, start_time). Experiment 1: ALL → range, 60,261 → 3,259 rows, 16.6 → 0.46 ms.

Q7: GROUP BY, HAVING vs. WHERE. WHERE filters individual rows before grouping and cannot reference aggregates. GROUP BY buckets the survivors into one output row per group. HAVING then filters whole groups after aggregation and may reference aggregates — an aggregate in WHERE is a syntax error. "Months with $\geq$ 1,000 completed trips" shows the split: WHERE discards non-completed rows, GROUP BY buckets by month, HAVING keeps only buckets meeting the threshold. The logical order is FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

Q8: VIEW vs. STORED PROCEDURE. A view is a saved SELECT that behaves like a table but stores no data; it is expanded at query time. It suits query reuse, column-level access control, and insulation from schema changes. A stored procedure is a precompiled server-side program with parameters and control flow, invoked with CALL; its body contains semicolons, hence the DELIMITER switch. In our project the finance view `v_completed_revenue` wraps the daily revenue query, while `sp_check_unpaid(24)` parameterizes the collections job. A view is queried; a procedure is run.

5 Evaluation

5.1 Workload Profile

The workload splits along the lines argued in Section 1: dispatch and payment writes need strong consistency, while reporting queries can tolerate weaker freshness. Q6 (hourly dispatch), Q1 (daily reconciliation), Q5 (periodic collections), Q2 (collections by driver) and Q7 (monthly reporting) are the five most frequent queries. All ten queries in Section 4 are read-only SELECTs; writes go to `Trip` and `Payment` at dispatch and settlement time.

5.2 Experiment 1: Index Before/After

We picked a query that counts completed trips since 2026-10, combining a `status` equality check with a `start_time` range condition. With no secondary index on those columns the plan is a full table scan (`type=ALL`, rows ≈60,261). After building a composite index on `(status, start_time)` — the equality column first, the range column second, following the leftmost-prefix rule — the plan switches to an index range scan. The leaves already contain everything the query needs, so there is no table lookup at all (`Using index`).

Table 5: Index before/after on the 60,000-row `Trip` table.

	`type`	`key`	est. rows	actual time
before (no index)	ALL	—	60,261	16.6 ms
after (composite)	range	`idx_trip_status_time`	3,259	0.46 ms

The same session also showed the non-sargable trap. Wrapping the indexed column in a function (`DATE(start_time) = '2026-10-15'`) forced a sequential scan of the whole index, while writing the condition as a half-open interval brought `type` back to `range`. Against the latency target G2 (P95 $<$ 200 ms), the indexed plan sits two orders of magnitude below the bar.

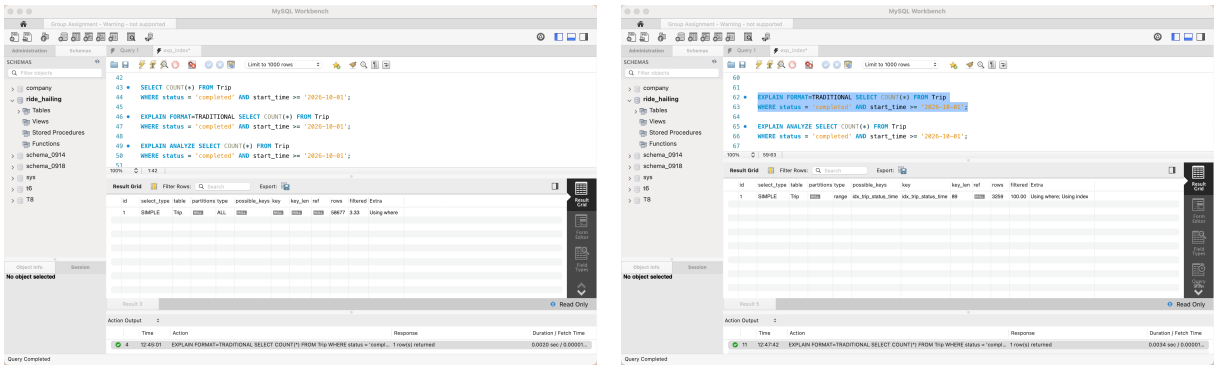


Figure 4: Experiment 1 before/after: `type` ALL → range, `rows` 60,261 → 3,259.

5.3 Experiment 2: JOIN vs Subquery

"Completed trips without payment" (Q2 and Q5) can be written three ways: `NOT EXISTS`, `NOT IN`, and `LEFT JOIN ... IS NULL`. All three returned the same 1,442 rows on our data, and the plans came out identical: a full scan of `Trip`, each row probed against the unique index on `Payment.trip_id` (`eq_ref`), with `Extra: Not exists; Using index` — the optimizer had rewritten both subquery forms into the same anti-join as the explicit LEFT JOIN version.

Table 6: Three anti-join formulations on the same question.

Formulation	result	plan signature	actual time
`NOT EXISTS`	1,442	`Not exists; Using index`	89 ms
`NOT IN`	1,442	same as above	82 ms
`LEFT JOIN ... IS NULL`	1,442	same type, minor `filtered` diff	90 ms

`NOT IN` carries a caveat: the subquery column has to be NOT NULL for it to work correctly. `Payment.trip_id` is, so all three are safe here; but if it allowed NULL the whole result would silently come back empty. That is the practical reason to check EXPLAIN before committing to one formulation (see Q4 in Section 4).

5.4 Experiment 3: Concurrent Transactions

We used two Workbench windows connected to the same database, operating on row `trip_id=92` of `trip_detail` (initial rating 3).

In the first scenario both sessions read the rating into their own variables, then wrote back computed values. Session A stored 3, added 1, and committed a 4. Session B, still holding its copy of 3, subtracted 1 and committed a 2 — overwriting A's 4. Both UPDATE statements reported "1 row(s) affected", yet A's increment was gone.

The second scenario replayed the same steps, except A locked the row with `SELECT ... FOR UPDATE` before writing. B's identical `FOR UPDATE` statement now blocked until A committed. Once unblocked, B read the committed value (3, not the stale copy) and added 1 again. Both increments survived: $2 \rightarrow 3 \rightarrow 4$.

For the third scenario we switched to row `trip_id=150`. Under the default REPEATABLE READ, a transaction that read the rating as 3 kept seeing 3 even after another session committed a 5 — the snapshot does not update mid-transaction. Under READ COMMITTED the same sequence saw the committed 5, then saw the 3 after a second

commit. This is exactly why plain SELECTs were not enough in scenario 2, and why `FOR UPDATE` was needed there.

Table 7: Concurrency scenarios and constraint interceptions.

Scenario	Observation	What it shows
Lost update	final rating 2, one write lost	the failure mode
`FOR UPDATE`	B waits, then $2 \to 3 \to 4$	row lock fixes it
RR re-read	still 3 after external commit	snapshot read
RC re-read	sees 5, then sees 3	reads latest commit
FK to missing trip	`ERROR 1452`	referential integrity
Duplicate `trip_id` payment	`ERROR 1062`	1:1 via UNIQUE
Rating out of 1–5 range	`ERROR 3819`	CHECK constraint

The last three rows also serve as evidence for G5: the constraints themselves stop illegal writes at the schema layer, without any application code involved.

5.5 Distribution Check against the RideAustin Holdout

The generator parameters came from the 80% training split of RideAustin; we then compared the generated 60,000 trips against the held-out 20%.

Table 8: Generated sample vs. RideAustin holdout.

Dimension	RideAustin	Generated
Tier mix (R/S/P/L)	95.0 / 3.7 / 0.9 / 0.3 %	95.1 / 3.7 / 0.9 / 0.3 %
Mean fare (completed)	\$13.99	\$14.00
Rating coverage	≈90.7 %	89.9 %
Evening peak 17–20h share	≈20.5 %	22.4 %

No dimension deviates by more than two percentage points. What this validates is distribution consistency, not the external validity of business conclusions. Status mix, payment methods and the unpaid rate follow course-defined rules with no counterpart in the source data, so we left them out of the table.

5.6 Results Summary and Goal Recheck

The composite index brought estimated rows down from 60,261 to 3,259 and actual latency from 16.6 ms to 0.46 ms, roughly 36 times faster and well inside the 200 ms P95 target. The three anti-join formulations produced identical results and near-identical plans, confirming that the optimizer, not the syntax, decides performance. On concurrency, the lost update turned out to be a read-modify-write problem that `FOR UPDATE` solved; the three error codes (1452, 1062, 3819) showed the schema constraints working on their own. These results, together with the holdout check, address the measurable goals set in Section 1.

6 Conclusion and Limitations

6.1 Goal Recheck

Table 9 closes the five quantified goals of Section 1.3 one by one, each row keeping the verification calibre declared in Table 2: an experiment measurement of Section 5 where one exists, a declared design-side claim where it does not—nothing is upgraded in the retelling.

Table 9: Goal recheck against the quantified goals of Section 1.3 (Table 2)

#	Outcome	Evidence
G1	delivered: 60,000 trips (target $\geq$ 50,000)	import row-count self-checks; clean re-run on a second machine, all self-checks green
G2	single-run query meets the target value: 0.46 ms after indexing vs 16.6 ms before (two orders below the 200 ms bar); **P95 pending measurement**	Experiment 1, single-run EXPLAIN ANALYZE magnitude on the Q5 predicate
G3	design estimate only: ≈36–72× write headroom over the 0.69–1.4 TPS peak model; **throughput pending measurement**	declared design-margin claim, not an experiment acceptance item
G4	1,442 completed-without-payment trips isolated, identical row set across all three anti-join forms, at 82–90 ms inside the 500 ms full-run budget; cross-check against the generator's expected unpaid-trip list **pending**	Q2/Q5 anti-join on the full table, Experiment 2
G5	met: illegal writes rejected by the database itself, zero orphans by the same declarative keys	FK error 1452, UNIQUE 1062, CHECK 3819 demonstrated in Experiment 3

6.2 Limitations

Four boundaries should be read alongside these results. First, the dataset is synthetic: the distributional agreement with the RideAustin holdout (Section 5) reflects the generator's calibration rules rather than real-world fit, so we report similarity and make no causal claims; the 15-zone pickup distribution in particular is a declared course setting without a calibration source. Second, by decision D-05 a trip enters the database only when dispatch succeeds, so unmatched requests are structurally invisible: order volume, dispatch success rate and matching latency cannot be analysed in this schema. Third, all performance figures are single-run magnitudes on one laptop-scale MySQL instance at 60,000 rows; they are order-of-magnitude evidence, not measured percentiles, and do not extrapolate to production scale or hardware—on this single instance the read-replica tier of the consistency design (Section 1.3) remains a declared stance rather than a deployed

component. Fourth, the sample is thin in its tails: with 150 drivers and ≈180 LUXURY trips, per-tier and per-driver aggregates are illustrative rather than statistically significant.

6.3 Future Work

Three extensions follow naturally. Partitioning `Trip` by `start_time` would turn the 180-day retention window into a drop-partition archival job instead of row-by-row deletes, and pairs well with the composite index of Experiment 1—at a real cost: MySQL 8.0 allows no foreign keys on partitioned InnoDB tables, and `Trip` sits on both sides (its own FKs, plus `Payment` and `trip_detail` referencing it), so the gain would be paid for in declarative referential integrity, re-established in the application layer. Deploying the declared read replica would route the wide aggregate reads (reconciliation reports, monthly revenue, the dispatch dashboard) away from the write-heavy primary. Finally, a pre-insert matching-event table (request, driver offer, dispatch outcome) would close the D-05 scope gap and bring dispatch analytics within reach of the same five-entity core; re-modelling that event stream against a document store (NoSQL comparison) would be the natural next experiment.

References

[1] CDS 534 Teaching Team. Cds 534 database management group project requirement. Course handout, 16 pp. (group shared drive), 2026. Official DDL, queries, and grading rubric for Topic 5.

[2] New York City Taxi and Limousine Commission. Tlc trip record data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page, 2026. Accessed 2026-10-22. High Volume FHV monthly records and taxi-zone lookup table, used only as a distributional reference for our synthetic generator (decision D-10); the page carries no data-specific open license, and NYC.gov Terms of Use apply.

[3] Uber Engineering. Why uber engineering switched from postgres to mysql. Uber Engineering Blog, https://eng.uber.com/postgres-to-mysql-migration/, 2016. Recorded as source 2 of the A1 business-flow memo (2026-10-22); core-transaction store practice cited in §1: every charge must be a durable transactional record.

[4] Uber Technologies. Am i charged for cancelling a trip? (uber help). https://help.uber.com/en/riders/article/am-i-charged-for-cancelling-a-trip-?nodeId=5f6415dc-dfdb-4d64-927a-66bb06bc4f82, 2026. Accessed 2026-10-22. First-party wording on cancellation-fee trigger and purpose; the real grace period and fee vary by city — our course simplification is a flat USD 1.50 fee (decision D-06 v1.2; an early CNY 8 draft value was superseded by the group's 10-23 currency vote).

[5] Natalia Zuniga-Garcia, Natalia Ruiz Juri, Randy Machemehl, James Scott, and Mauricio Tec. Evaluation of ride-sourcing search frictions and driver productivity: A spatial denoising approach. RideAustin rides dataset, Mendeley Data, V1, 2019. Approx. 1.49M ride-sourcing trips, 2016-06-02 – 2017-04-13. License CC0 1.0. Distributional calibration source for our synthetic generator (decision D-10): tier mix, per-trip ratings (92.6% non-null), fare rates. Verified 2026-10-22.

[6] Natalia Zuniga-Garcia, Natalia Ruiz Juri, Randy Machemehl, James Scott, and Mauricio Tec. Evaluation of ride-sourcing search frictions and driver productivity: A spatial denoising approach. *Transportation Research Part C: Emerging Technologies*, 2019. Accessed 2026-10-22.

A Contribution Statement

Member (SID)	Role	Main Contributions	Load
CHEN Yinan (3162550)	A	Business modelling: process flow, stakeholder needs, quantified goals G1–G5 and constraints; dataset selection and RideAustin calibration profile; billing and cancellation rules (D-05/D-06); workload profile and consistency trade-offs; author of Section 1	15%
ZHANG Yijie (3162641)	B	ER model v1.0.1 and vector diagram; data dictionary (8 tables, 26 columns); extension proposal D-11; normalization review; ER–DDL reconciliation and signed handoff; author of Section 2 with course-notation figure; Q1/Q2 reviewer	20%
CHEN Yanwu (3162744)	C	`build.sql` (official base + extension zone + self-checks); `import_data.py` with fixed seed and RideAustin calibration (60,000 trips); clean re-run evidence on a second machine (C8); schema v1.0; engineering disciplines D-07/D-08; author of Section 3 and Appendix B	20%
LING Tong (3162677)	D	Query catalogue q01–q10 with interpretations; eight concept answer cards; three experiments (index, JOIN vs subquery, concurrency) with archived screenshots; holdout comparison table; author of Sections 4–5	25%
HU Zhongyuan (3162304)	E	Project management: timeline, milestones and follow-ups; decision registry D-05–D-12; meeting minutes (kickoff, M1, M3); four quality gates (ER / SQL / numbers / format); cover, contribution table and Section 6 co-author; integration and final compilation	20%

Table 10: Team contributions (percentages pending group confirmation)

B Full Build Script

Data Generation Entry

The dataset is regenerated by `import_data.py` (repository path `02_sql/`): every random choice uses the fixed seed 534, distribution parameters are the RideAustin-calibrated

values of decision D-10, and the script ends with built-in acceptance checks (row counts and key ratios), so any member rebuilds identical data from the same repository scripts (constraint C6). Full rebuild on a clean machine:

Listing 11: Rebuild from scratch (idempotent, re-runnable)

```
mysql -u <user> -p < 02_sql/build.sql     # schema v1.0: base + extension + self-checks
python3 02_sql/import_data.py             # fixed seed=534 + built-in acceptance checks
```

Listing 12: build.sql — full DDL with repeatability self-check (schema v1.0 snapshot)

```
-- ============================================================
-- build.sql -- CDS534 group project, Topic 5: full database build script
-- Project: Ride-Hailing Order and Trip Dispatch database
-- (re-runnable: DROP + CREATE restores a clean, identical schema)
--
-- Usage: MySQL Workbench: open this file and execute all (lightning button),
--        or command line: mysql -u <user> -p < build.sql
--
-- Base rule: Section 1 is the official Topic 5 DDL (course handout Section 5,
-- Listing 17; only PDF layout whitespace cleaned, semantics identical). It is
-- never modified; extension tables are appended in Section 3 only.
--
-- Version: schema v1.0 (finalised 2026-11-05; ER v1.0.1 frozen and the
-- ER-DDL reconciliation closed). Structure is unchanged from v1.0-rc; only
-- comments and the version tag were updated.
--
-- Version policy: no forced upgrades; compatibility baseline = the lowest
-- registered member version (8.0.45, see env_record.md). Collation
-- utf8mb4_general_ci works on all versions. Below 8.0, CHECK constraints
-- would be silently ignored (still runs); queries avoid window functions
-- and CTEs in favour of JOIN / subquery forms.
-- Maintainer: C (implementation lead); changes must be committed to GitHub.
--
-- NOTE (display copy): this file is the appendix rendering of 02_sql/build.sql
-- with comments translated to English (report language). DDL and SQL
-- statements are byte-identical to 02_sql/build.sql (schema v1.0).
-- ============================================================

-- 0. Dedicated project database (isolated from personal schemas; group-wide name)
DROP DATABASE IF EXISTS ride_hailing;
CREATE DATABASE ride_hailing
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci;
USE ride_hailing;

-- ============================================================
-- 1. Official base DDL (Listing 17, kept verbatim -- do not modify)
--    Five entities: Passenger / Vehicle / Driver / Trip / Payment
--    Key constraints: Driver.vehicle_id NOT NULL UNIQUE -- each driver is
--    bound to exactly one vehicle, and UNIQUE additionally caps each vehicle
--    at most one driver (the 1:1 on the ER diagram is maximum pairing; the
--    1:N arrow in the course figure is the business direction, unreachable
--    in practice). Payment.trip_id NOT NULL UNIQUE -- each trip has at most
--    one payment, and unpaid trips may exist (exactly what the official Q2
--    anti-join relies on).
-- ============================================================

CREATE TABLE Passenger (
    psg_id INT PRIMARY KEY,
```

```
    name    VARCHAR(40) NOT NULL
);

CREATE TABLE Vehicle (
    vehicle_id INT PRIMARY KEY,
    plate      VARCHAR(20) NOT NULL UNIQUE
);

CREATE TABLE Driver (
    driver_id  INT PRIMARY KEY,
    name       VARCHAR(40) NOT NULL,
    vehicle_id INT NOT NULL UNIQUE,
    FOREIGN KEY (vehicle_id) REFERENCES Vehicle (vehicle_id)
);

CREATE TABLE Trip (
    trip_id    INT PRIMARY KEY,
    psg_id     INT NOT NULL,
    driver_id  INT NOT NULL,
    fare       DECIMAL(8,2),
    status     VARCHAR(20),
    start_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (psg_id)    REFERENCES Passenger (psg_id),
    FOREIGN KEY (driver_id) REFERENCES Driver (driver_id)
);

CREATE TABLE Payment (
    pay_id  INT PRIMARY KEY,
    trip_id INT NOT NULL UNIQUE,
    amount  DECIMAL(8,2) CHECK (amount >= 0),
    method  VARCHAR(20),
    FOREIGN KEY (trip_id) REFERENCES Trip (trip_id)
);

-- ============================================================
-- 2. Official three queries (Listing 18-20, base verification)
--    On a fresh empty database, empty result sets already count as
--    "runs through" (syntax, constraints and joins are valid).
-- ============================================================

-- Q1 completed trips and their payments (1:1 join)
SELECT t.trip_id, p.name AS passenger, t.fare, py.amount, py.method
FROM Trip t
JOIN Passenger p ON p.psg_id = t.psg_id
JOIN Payment py  ON py.trip_id = t.trip_id
WHERE t.status = 'completed';

-- Q2 drivers of completed trips not yet paid (NOT EXISTS anti-join)
SELECT d.name
FROM Driver d
JOIN Trip t ON t.driver_id = d.driver_id
WHERE t.status = 'completed'
  AND NOT EXISTS (SELECT 1 FROM Payment py WHERE py.trip_id = t.trip_id);

-- Q3 cumulative income per vehicle
SELECT v.plate, SUM(t.fare) AS earned
FROM Vehicle v
JOIN Driver d ON d.vehicle_id = v.vehicle_id
JOIN Trip t   ON t.driver_id = d.driver_id
GROUP BY v.vehicle_id, v.plate;

-- ============================================================
-- 3. Extension zone (C3, schema v1.0; built 2026-10-25, finalised 11-05)
```

```
113 --     Shape per D-11 (both extensions adopted) + B4 data dictionary:
114 --     extension attributes live in separate tables keyed by trip_id; the
115 --     official five tables gain no columns (base rule).
116 --
117 --     ER cardinality <-> SQL constraint mapping:
118 --       ServiceTier 1-N Trip      <-> trip_detail.service_tier_id NOT NULL + FK
119 --       PickupZone   1-N Trip      <-> trip_detail.zone_id NOT NULL + FK
120 --       Trip         1-1 TripDetail <-> trip_detail.trip_id is both PK and FK
121 --                                       (PK-is-FK -> at most one extension row
122 --                                       per Trip)
123 --       Cardinalities inside the official five tables are unchanged (1:1 on
124 --       the diagram = maximum pairing; optionality is what UNIQUE encodes):
125 --       Vehicle 1:1 Driver via vehicle_id UNIQUE -> each vehicle has 0..1
126 --       drivers; Trip 1:1 Payment via trip_id UNIQUE -> each trip has 0..1
127 --       payments (the Q2 anti-join needs unpaid rows to exist).
128 --
129 --     Generator-side invariants (import_data.py / C4; not schema-enforced):
130 --       (1) every Trip has exactly one trip_detail row, cancelled rows
131 --           included: tier and zone are request attributes fixed at dispatch
132 --           (q09 counts all Trip rows per D-11 item 1);
133 --       (2) rating is non-NULL only for status='completed' rows (~90%
134 --           coverage, D-11 item 2; AVG ignores NULLs automatically). This
135 --           rule is cross-table semantics: a CHECK cannot reference another
136 --           table's column and triggers are beyond course scope, so the
137 --           generator guarantees it and the B4 data dictionary records it
138 --           (discussed in the report as a boundary of declarative constraint
139 --           expressiveness).
140 --
141 --     Dimension seeds (4 + 15 rows) ship with build.sql rather than the
142 --     import script: tier and zone names are course settings (D-11), part of
143 --     schema semantics rather than workload data; identical across machines
144 --     without Python, and visible in C5 screenshots on an empty database.
145 -- ============================================================
146
147 -- 3.1 Service tier dictionary (names keep RideAustin's native English terms;
148 --      do not translate to local ride-hailing product names)
149 --      Calibrated mix: REGULAR ~95.0% / SUV ~3.7% / PREMIUM ~0.9% /
150 --      LUXURY ~0.3% (RideAustin full-data measurement). fare_multiplier is a
151 --      descriptive column = measured relative multiple of mean total_fare;
152 --      generated fares use the four real rate cards directly (A4 section 3),
153 --      not this column.
154 CREATE TABLE service_tier (
155     tier_id          TINYINT UNSIGNED PRIMARY KEY,
156     name             VARCHAR(16)  NOT NULL UNIQUE,
157     fare_multiplier  DECIMAL(3,1) NOT NULL
158 );
159
160 INSERT INTO service_tier (tier_id, name, fare_multiplier) VALUES
161     (1, 'REGULAR', 1.0),
162     (2, 'SUV',     1.9),
163     (3, 'PREMIUM', 2.1),
164     (4, 'LUXURY',  2.7);
165
166 -- 3.2 Pickup zone dictionary (course setting: ~15 zones, top-3 ~35% share;
167 --      spatial concentration referenced from NYC TLC; RideAustin has no zone
168 --      field, hence no real calibration source -- the report declares the
169 --      zone distribution as a course setting per D-11 item 1. Names are
170 --      placeholders: rename freely, keep zone_id)
171 CREATE TABLE pickup_zone (
172     zone_id  TINYINT UNSIGNED PRIMARY KEY,
173     name     VARCHAR(32) NOT NULL UNIQUE
174 );
175
```

```sql
INSERT INTO pickup_zone (zone_id, name) VALUES
    (1,  'Downtown'),
    (2,  'East Austin'),
    (3,  'South Congress'),
    (4,  'South Lamar'),
    (5,  'West Campus'),
    (6,  'Riverside'),
    (7,  'Hyde Park'),
    (8,  'Mueller'),
    (9,  'Airport'),
    (10, 'North Loop'),
    (11, 'Crestview'),
    (12, 'Cherrywood'),
    (13, 'Barton Hills'),
    (14, 'Zilker'),
    (15, 'Tarrytown');

-- 3.3 Trip extension table (1:1: PK-is-FK, at most one row per Trip; carries
--     both D-11 extensions -- service tier x rating and pickup zone -- so
--     q09/q10 each need a single JOIN)
CREATE TABLE trip_detail (
    trip_id         INT PRIMARY KEY,              -- PK-is-FK -> 1:1 with Trip
    service_tier_id TINYINT UNSIGNED NOT NULL,    -- request attribute, fixed at dispatch
    zone_id         TINYINT UNSIGNED NOT NULL,    -- pickup zone (q09 hot-zone stats)
    rating          TINYINT UNSIGNED NULL,        -- passenger rates driver, 1-5 stars;
                                                   -- non-NULL only for completed rows (~90%)
    FOREIGN KEY (trip_id)         REFERENCES Trip (trip_id),
    FOREIGN KEY (service_tier_id) REFERENCES service_tier (tier_id),
    FOREIGN KEY (zone_id)         REFERENCES pickup_zone (zone_id),
    CONSTRAINT chk_rating CHECK (rating BETWEEN 1 AND 5)
);

-- ============================================================
-- 4. Self-check (for screenshots): version + table list + row counts
-- ============================================================
SELECT VERSION() AS mysql_version;
SHOW TABLES;
-- Row counts right after this script (before data import): base five
-- tables = 0; dimension tables already seeded = service_tier 4 /
-- pickup_zone 15; trip_detail is generated together with Trip by C4 = 0.
SELECT 'Passenger'   AS tbl, COUNT(*) AS rows_ FROM Passenger
UNION ALL SELECT 'Vehicle',      COUNT(*) FROM Vehicle
UNION ALL SELECT 'Driver',       COUNT(*) FROM Driver
UNION ALL SELECT 'Trip',         COUNT(*) FROM Trip
UNION ALL SELECT 'Payment',      COUNT(*) FROM Payment
UNION ALL SELECT 'service_tier', COUNT(*) FROM service_tier
UNION ALL SELECT 'pickup_zone',  COUNT(*) FROM pickup_zone
UNION ALL SELECT 'trip_detail',  COUNT(*) FROM trip_detail;
```